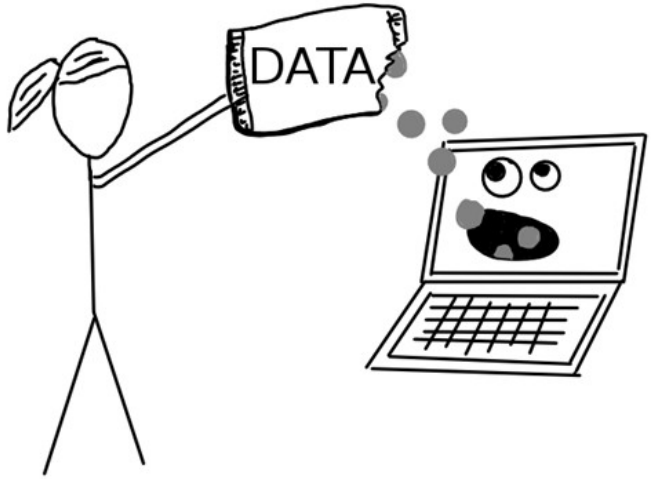
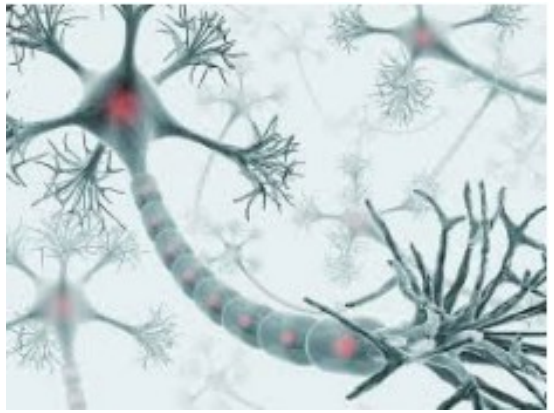
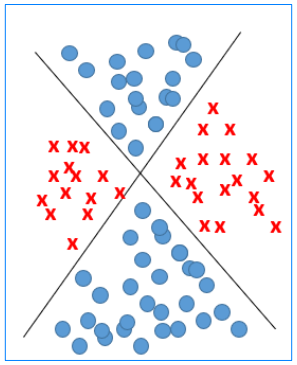






Multilayer Neural Network
non linearly separable problem

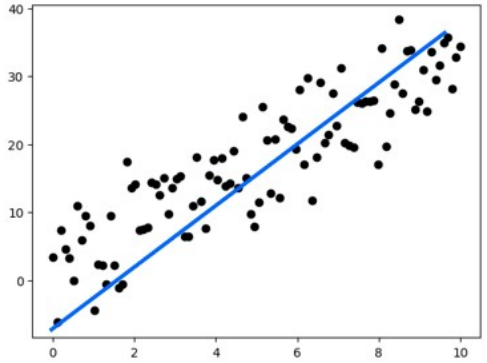
2022/2023

Abdelwahab Naji - Enseignant chercheur
Abdelwahab.naji@gmail.com
ENSAO, Université Mohamed premier - Oujda
https://www.youtube.com/c/AbdelwahabNaji

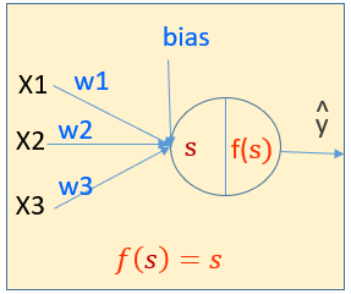
Neural Network for Supervised Learning

1

Simple Linear Regression Problem



Random point cloud

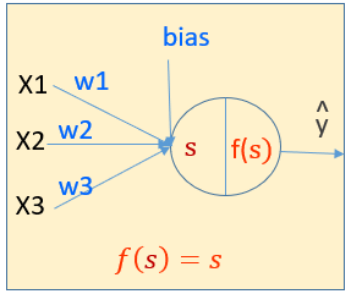


2

Multi Variate Linear Regression Problem

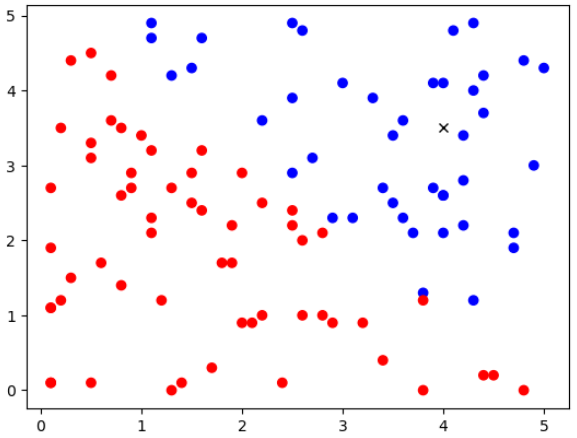
TV	Radio	Newspaper	CA
230.1	37.8	69.2	**22,1**
44.5	39.3	45.1	**10,4**
17.2	45.9	69.3	**9,3**
...			
199.8	2.6	21.2	**10,6**

Advertising p

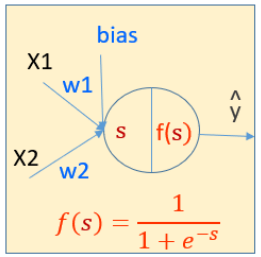


3

Linearly Separable problem

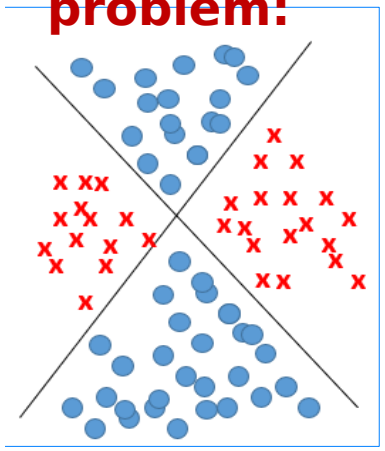


Red or Blue?

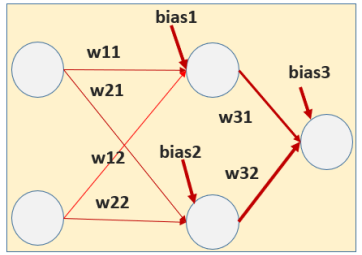


4

Non Linearly Separable problem:

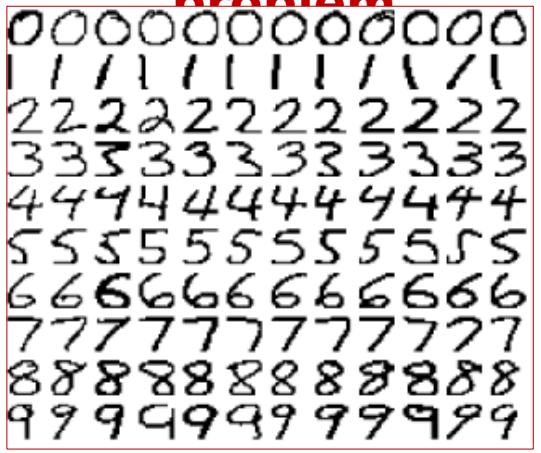


XOR Problem

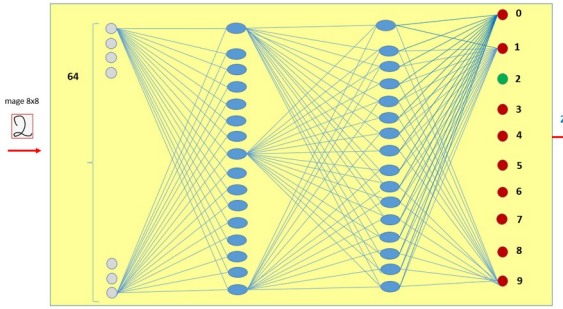


5

Complex classification problem

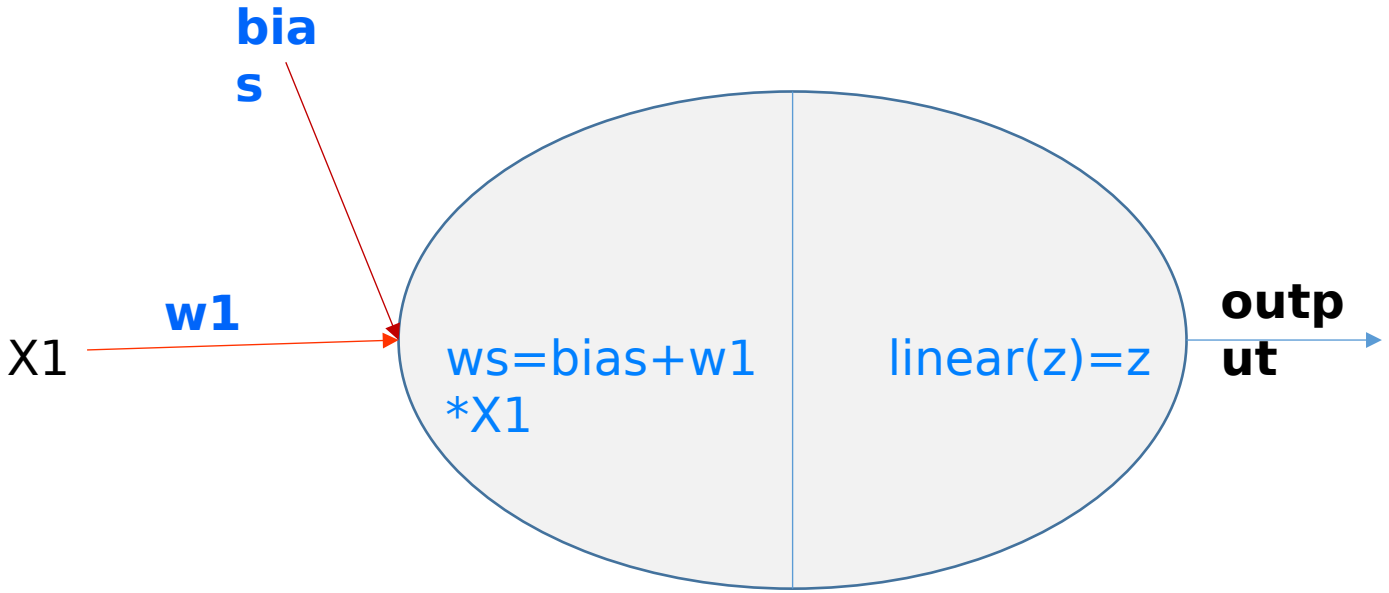


Hand Written digits



Neural Network – One Neurone

Simple Regression Problem



1

Simple Regression problem

```
#Librairies
import matplotlib.pyplot as plt
from random import seed
from random import
```

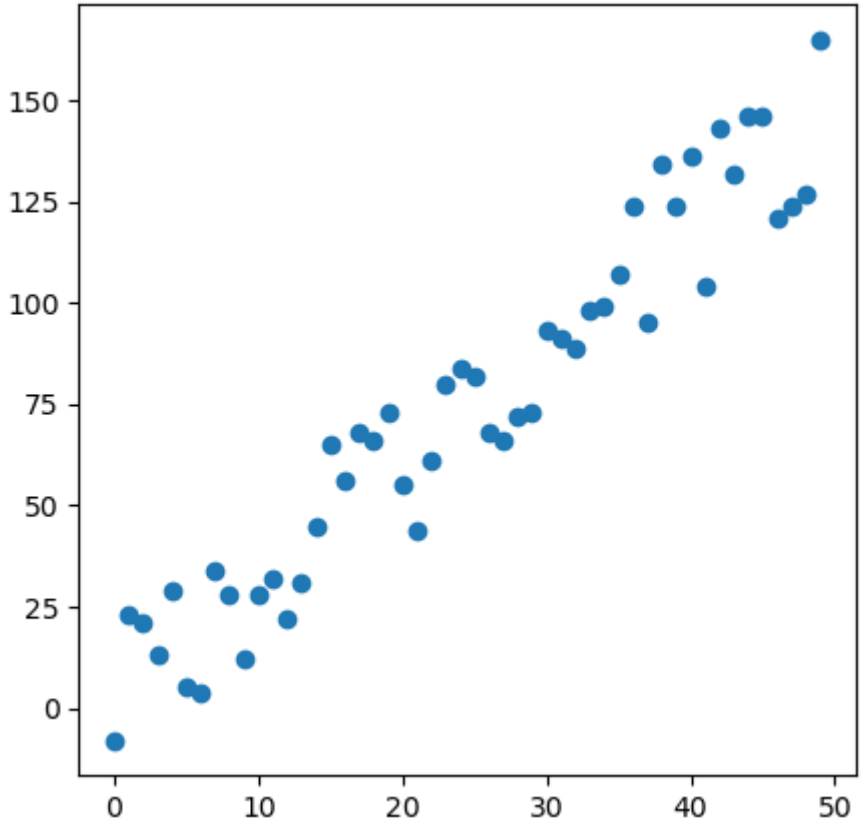
```
np.random.seed(0)
```

```
def f(X):
    return 1+3*X
```

```
def generateData (n):
    X=np.array([i for i in range(n)])
    y_model=f(X)
    y =np.array([i+randrange(-20,20) for i in y_model])
    return X,y_model,y
```

```
n_samples = 50
X,y_model,y=generateData(n_samples)
```

```
plt.figure(figsize=(5, 5))
plt.scatter(X, y)
plt.show()
```



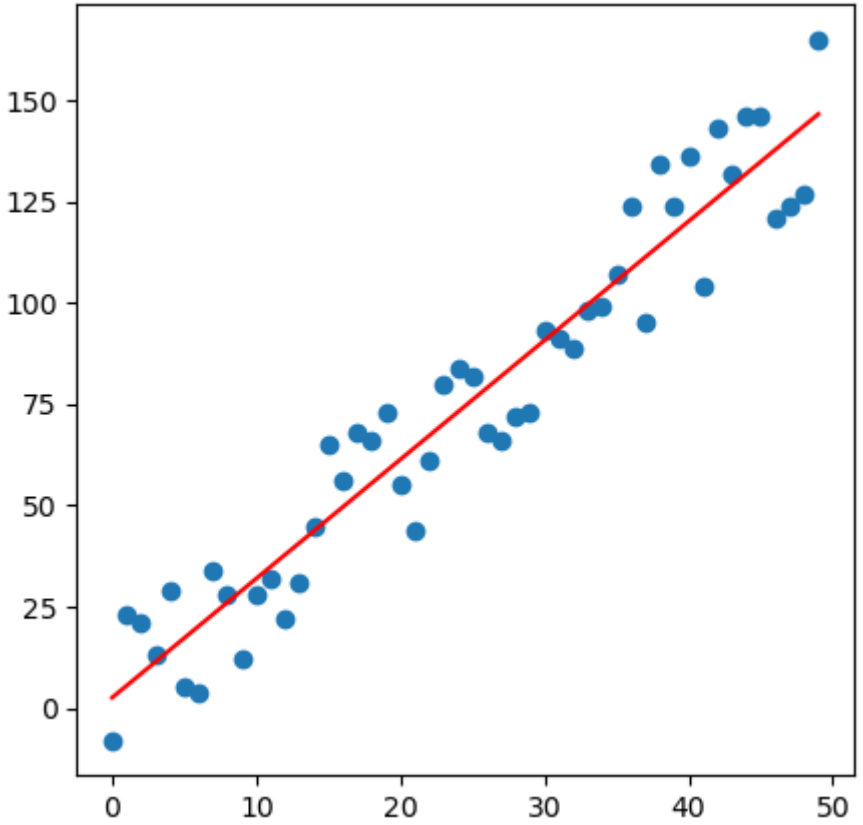
```
def fit(X,y):
    lr=0.001
    epochs=10000
    bias=0.5
    w1=0.5
    n=len(y)
    for i in range(epochs):
        py=bias+w1*X
        dbias=-(2/n)*sum(y-py)
        print(epochs,"---------------->",w1," , ",bias)
        dw1=-(2/n)*sum(X*(y-py))
        w1=w1-dw1*lr
        bias=bias-dbias*lr
    return bias,w1
```

```
bias,w1=fit(X, y)
```

```
def predict(X):
    return bias+w1*X
```

```
predicted_y=predict(X)
```

```
plt.figure(figsize=(5, 5))
plt.title("Simple Regression Model)", size=16)
plt.scatter(X, y)
plt.plot(X, predicted_y, c="red")
plt.show()
```



2

Simple classification problem Linearly Separable

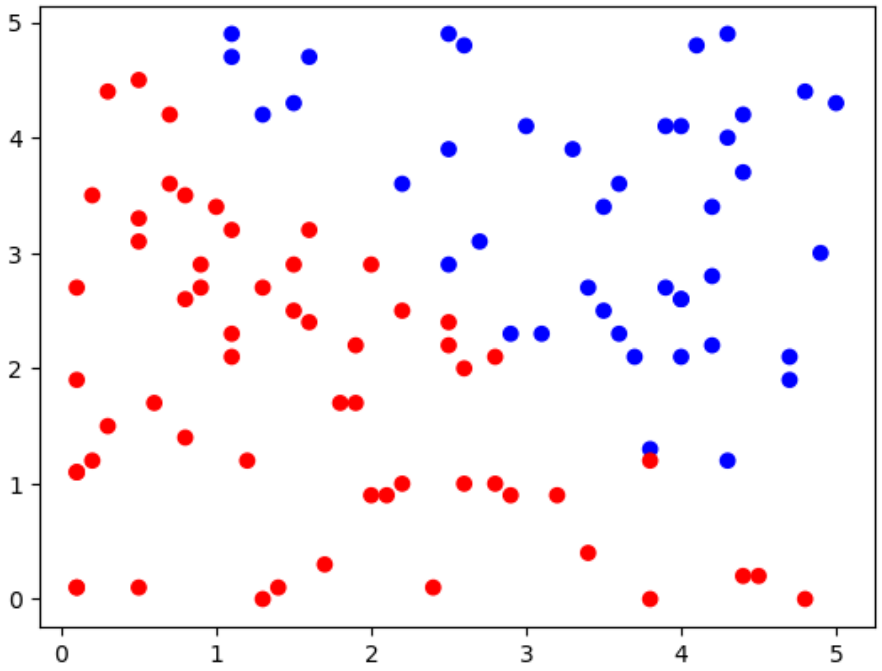
```
import matplotlib.pyplot as plt
import numpy as np
import matplotlib.colors as clr
```

```
def f(x):
    return -x+5
```

```
# generate random floating point values
from random import seed
from random import random
# seed random number generator
def generateData(n):
    seed(1)
    min=0
    max=5
    # generate random numbers between
0-1
    L=[]
    for _ in range(n):
        valueX = random()
        scaledvalueX = min + (valueX *
(max - min))
        valueY = random()
        scaledvalueY = min + (valueY *
(max - min))
        x=round(scaledvalueX,1)
        y=round(scaledvalueY,1)
        y0=f(x)
        cl=0
        if(y>y0):
            cl=1
```

```
L=generateData(10
0)
L=np.array(L)
X=L[:,0]
y=L[:,1]
```

```
colors = ['red','blue']
plt.scatter(L[:,0],L[:,1],c=L[:,2],cmap=clr.ListedColor
map(colors))
```



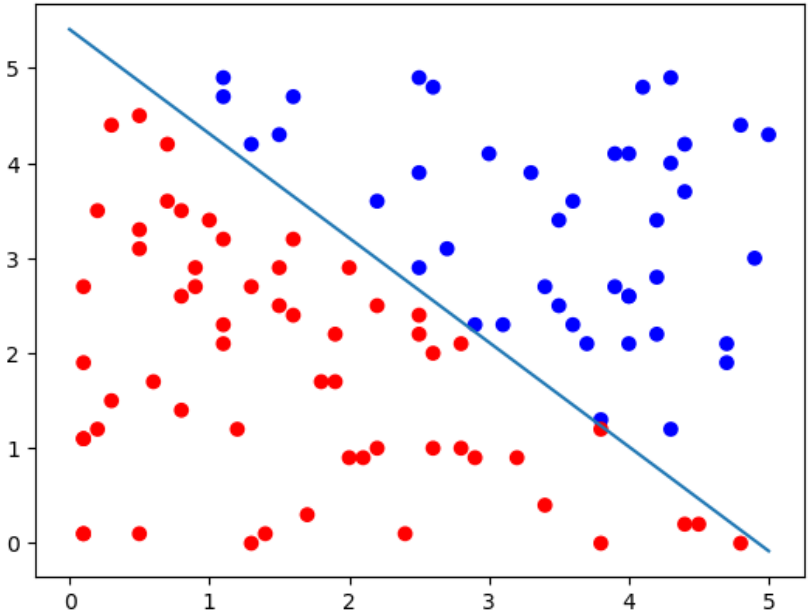
```
def fit(X,y):
    #initialisation
    n=len(y)
    bias=0.5
    w1=0.5
    w2=0.5
    epochs=5000#hyperparamter
    lr=0.01 #hyperparameter
    #learning
    for i in range(epochs):
        y=L[:,2]
        ws=bias+w1*L[:,0]+w2*L[:,1]
        py=sigmoid(ws)
        #update
        dbias=-(2/n)*sum(y-py)
        dw1=-(2/n)*sum(L[:,0]*(y-py))
        dw2=-(2/n)*sum(L[:,1]*(y-py))

        bias=bias-dbias*lr
        w1=w1-dw1*lr
        w2=w2-dw2*lr
    return bias,w1,w2
```

```
bias,w1,w2=fit(X,y)
```

```
def getY(x,bias,w1,w2):
    return -w1/w2*x-bias/w2
```

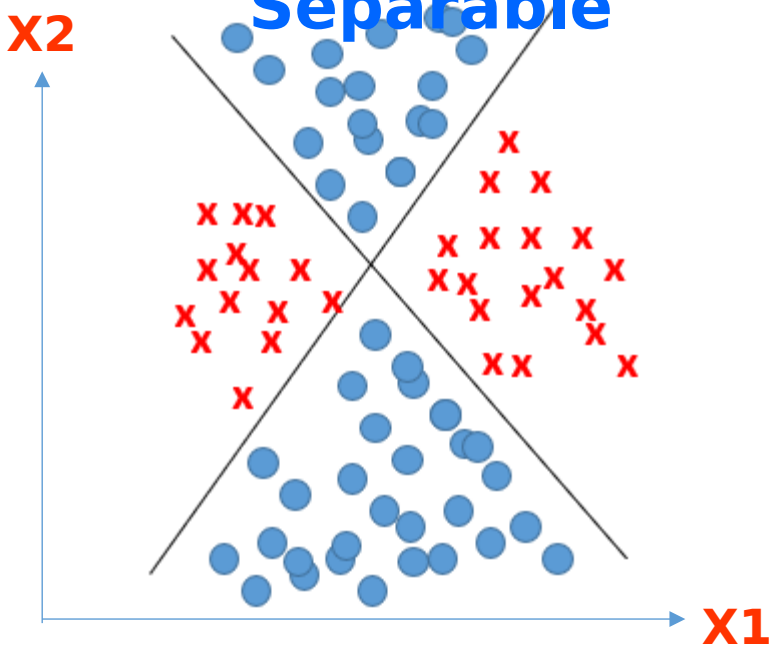
```
x1=0
x2=5
y1=getY(x1,bias,w1,w2)
y2=getY(x2,bias,w1,w2)
```

```
colors = ['red','blue']
plt.scatter(L[:,0],L[:,1],c=L[:,2],cmap=clr.ListedCol
ormap(colors))
#plt.plot([x1,x2],[y1,y2])
plt.plot([x1,x2],[y1,y2])
```



3

Complex classification problem non Linearly Separable



4

Images classification

Mise en œuvre de Neural Network

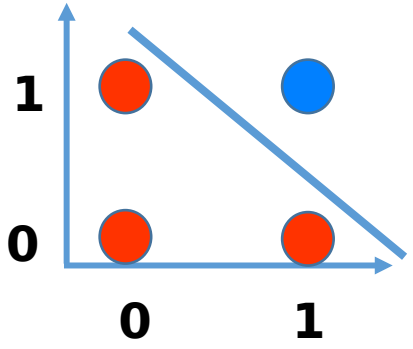
- **NN: Multi – Layer**
  - Présentation du problème **non linéairement séparable**
  - Implémentation du Problème **non linéairement séparable (**avec tensorflow, keras **)**
    - Exemple 1: simple XOR problem
      - Générer les données
      - Créer et entrainer un multilayer neuralnetwork
    - Exemple 2: general XOR problem
      - Générer les données
      - Créer et entrainer un multilayer neuralnetwork
    - Exemple 3: fashion classification problem
    - Exemple 4: hand written digits classification problem
  - Implémentation from scratch
    - Exemple: fashion classification problem

Problem

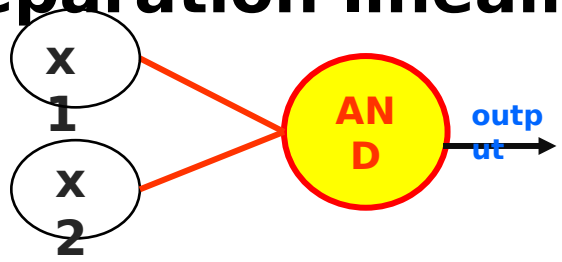
Séparation de
AND, OR,NAND, NOR, XOR

And

	F	T
T	F	T
F	F	F

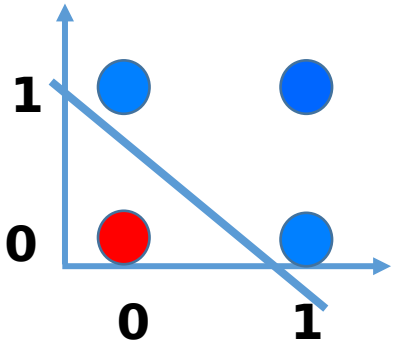


On peut avoir une séparation linéaire

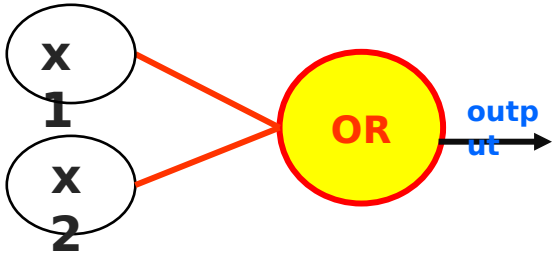


OR

	F	T
T	T	T
F	F	T

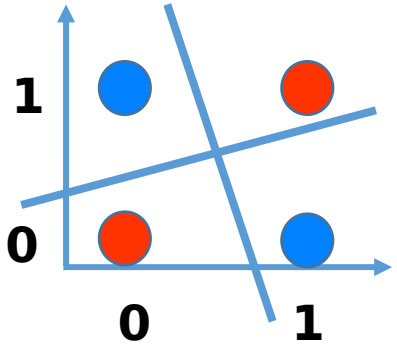


On peut avoir une séparation linéaire



XOR

	F	T
T	T	F
F	F	T

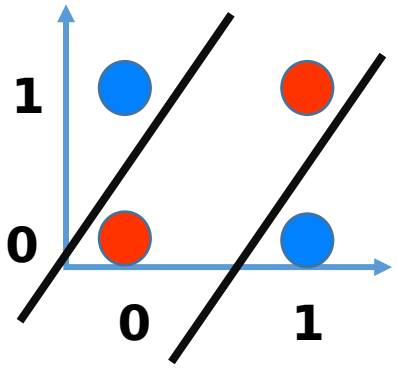


On ne peut pas avoir une séparation linéaire

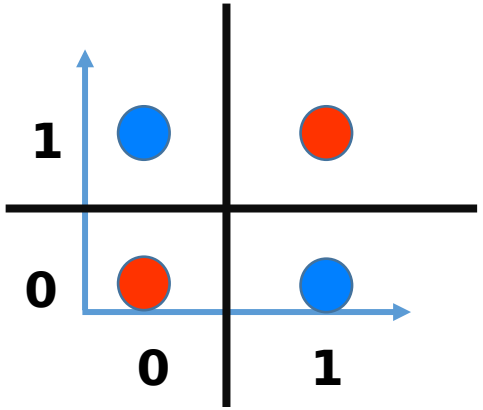
Séparation de X0R

XOR

	F	T
T	T	F
F	F	T

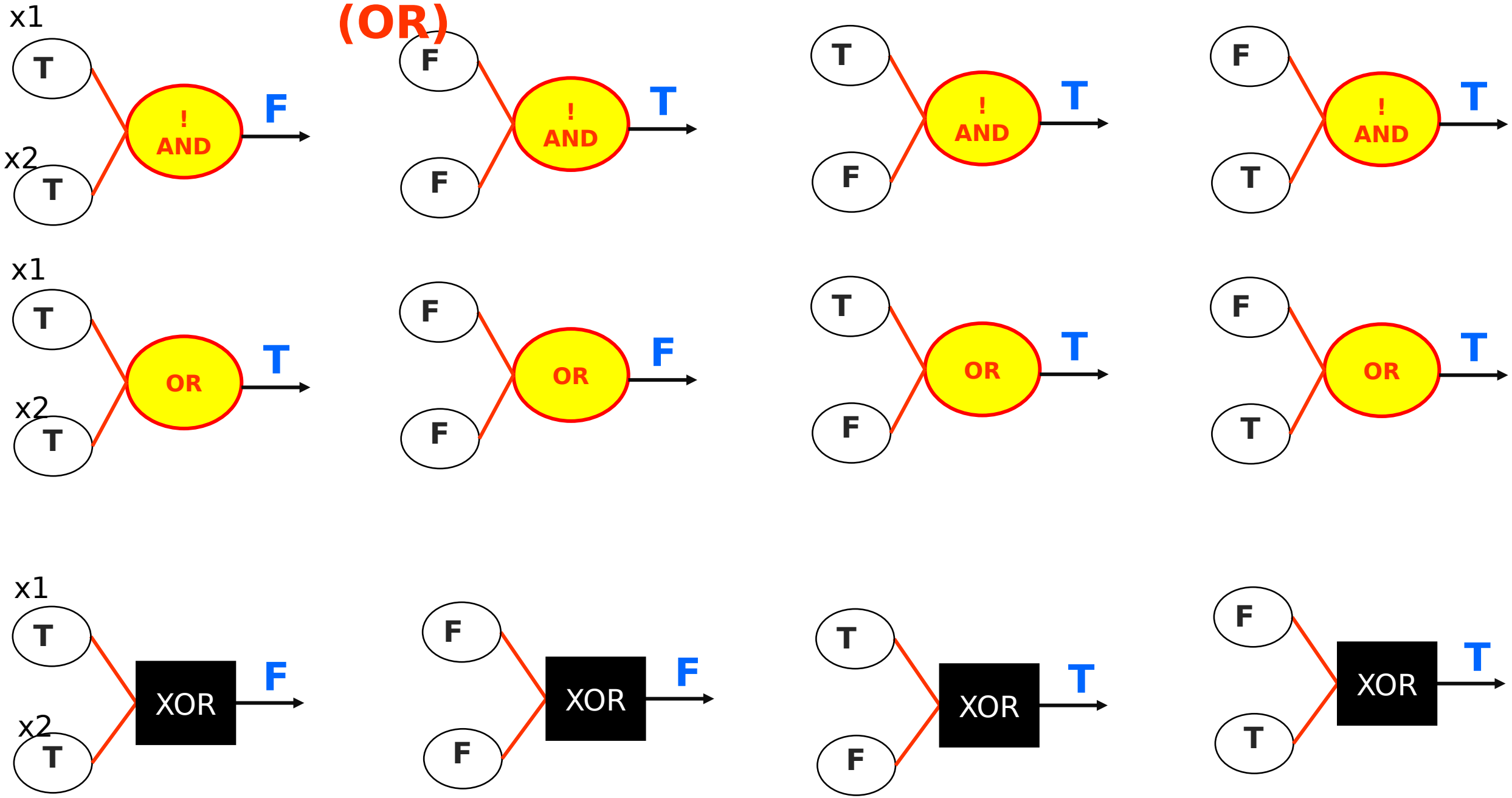


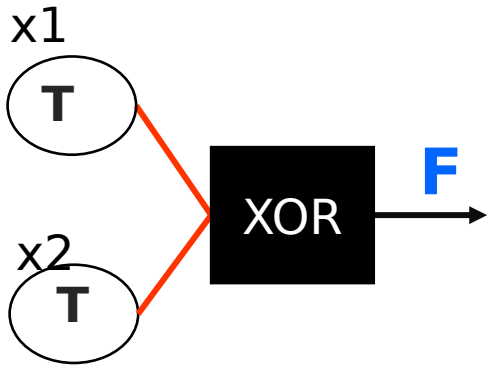
On peut utiliser plus qu'une ligne pour faire la séparation

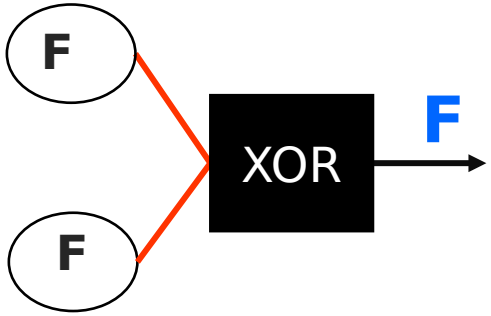


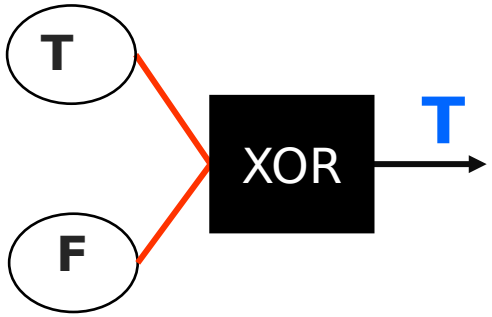
XOR = (!AND) and (OR)

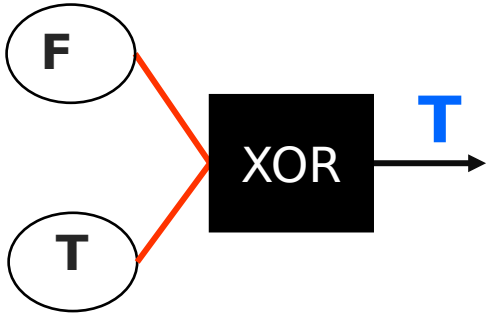
XOR = (!AND) and (OR)

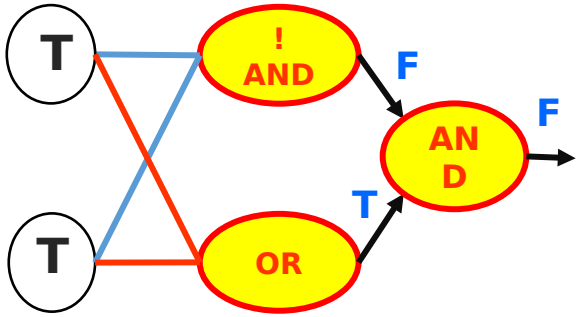


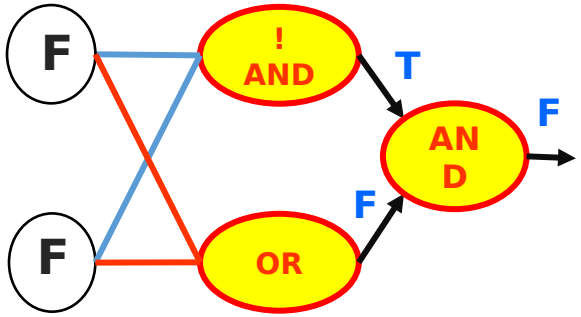

x1
T
x2
T
XOR
F

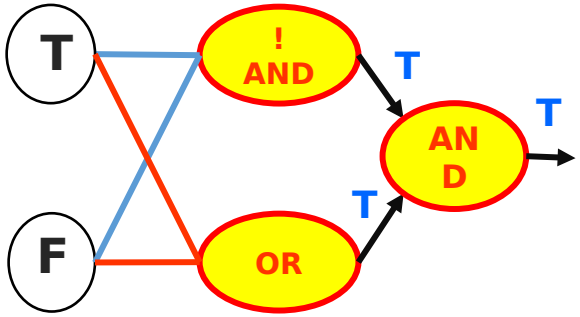

F
F
XOR
F

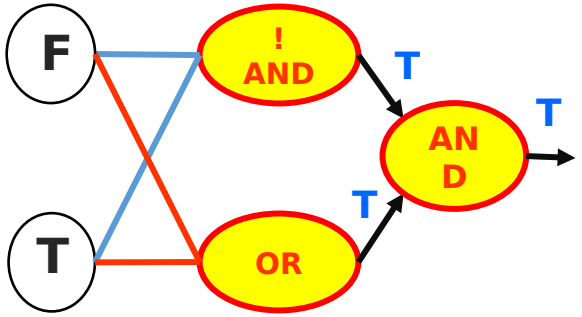

T
F
XOR
T

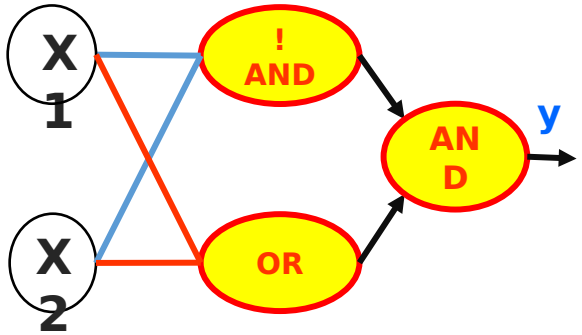

F
T
XOR
T


T
T
! AND
OR
F
T
AN D
F


F
F
! AND
OR
T
F
AN D
F


T
F
! AND
OR
T
T
AN D
T

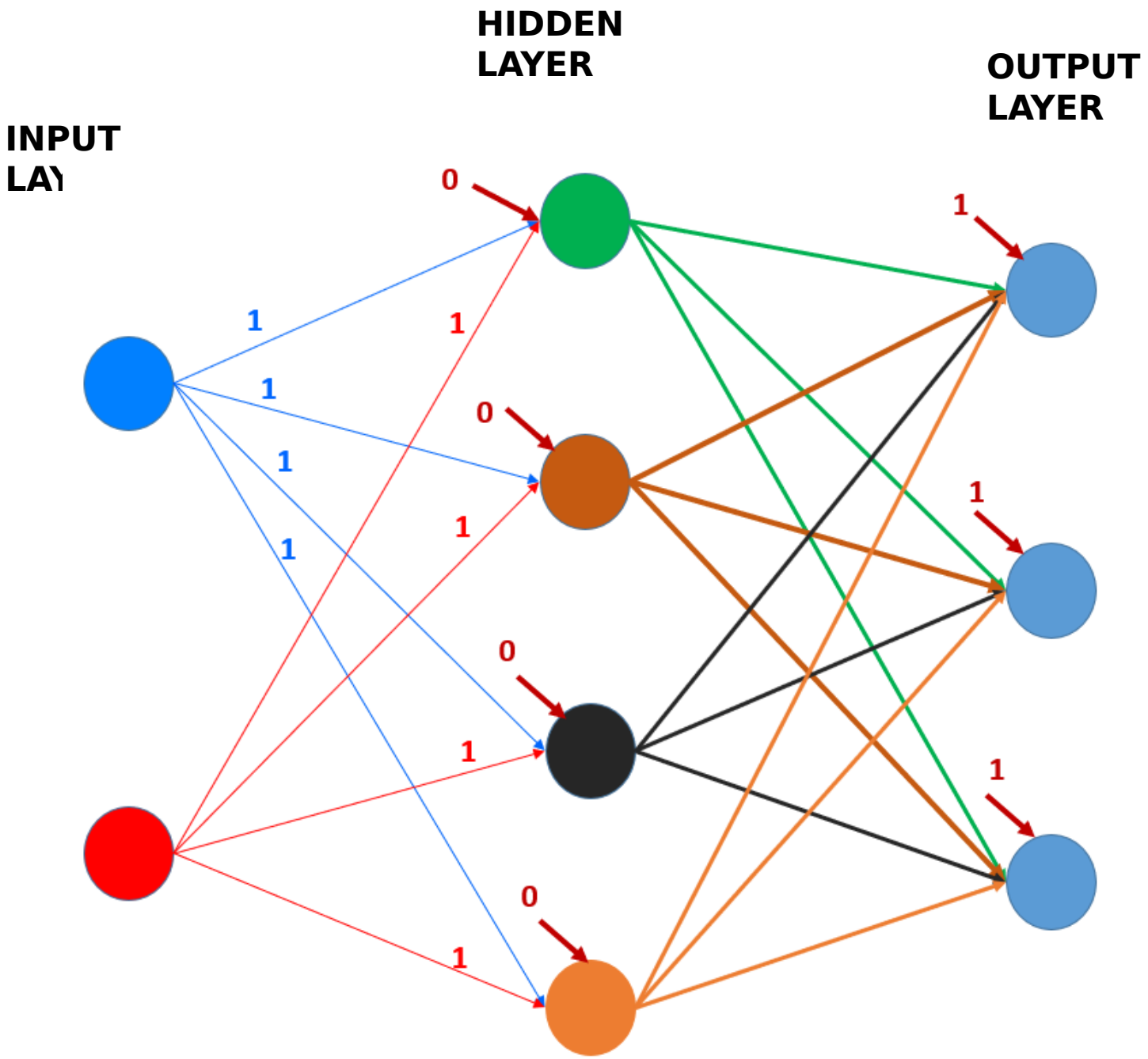

F
T
! AND
OR
T
T
AN D
T


X 1
X 2
! AND
OR
AN D
y

Utiliser keras pour créer un réseau de neurone avec plusieurs neurones (couche, layer)

Ce qu’il faut connaitre:

1. **Comment créer un multilayer neural network (en général)?**
2. Découvrir les éléments du neural network créé: coeficients, bias


HIDDEN LAYER
OUTPUT LAYER
INPUT LAY
0
1
0
1
0
1
1
1
1
1
1
1
1
1

Modèle et ses paramètres

1. Créer le modèle
2. Créer une couche et l’ajouter au modèle
3. Paramètres d’une couche

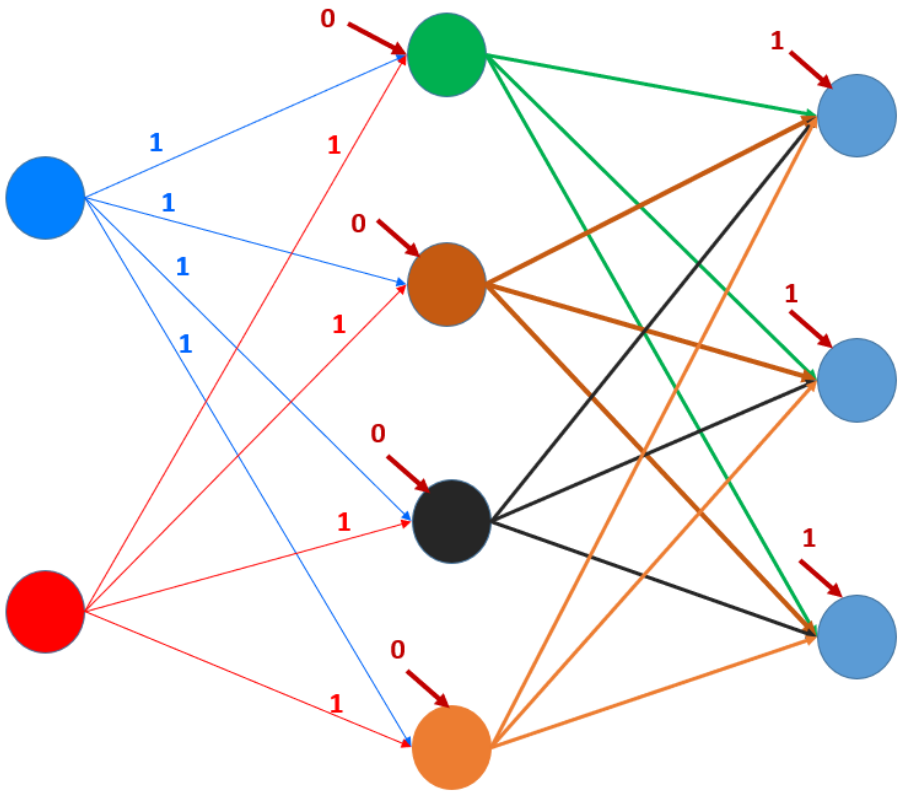
Model et ses paramètres

```
from keras.models import Sequential
from keras.layers import Flatten, Dense, Activation
```

```
model = Sequential()
```

```
#input_layer=Flatten(input_shape=[2, 1])
#model.add(input_layer)
hidden_layer=Dense(4,
input_dim=2,kernel_initializer='ones',bias_initializer='zeros
')
model.add(hidden_layer)
output_layer=Dense(3,
kernel_initializer='random_uniform',bias_initializer='ones')
model.add(output_layer)
print(model.summary())
```

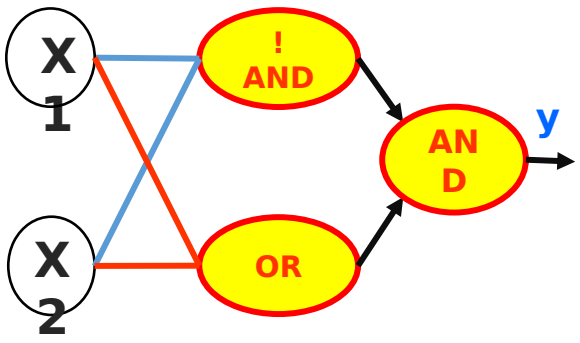
```
model.compile(loss="categorial_crossentropy",
            optimizer="adam",
            metrics=["accuracy"])
```



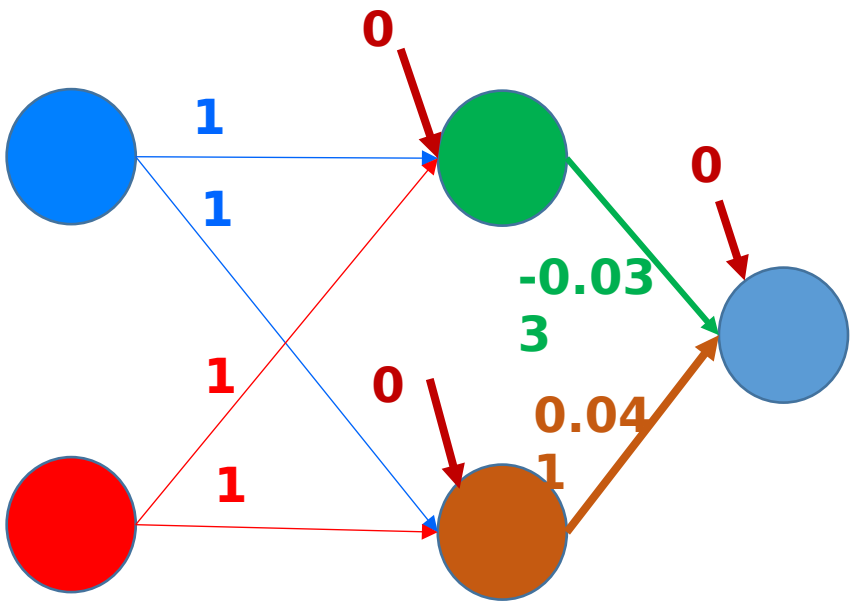

```
[array([[1., 1., 1., 1.],[1., 1., 1., 1.]],
dtype=float32),
       array([0., 0., 0., 0.], dtype=float32),
 array([[-0.03575971, -0.02119735,
0.01558626],
     [ 0.04222678, -0.04578313,  0.0360521 ],
     [-0.02753322,  0.00745941,
-0.04211374],
     [-0.04299896, -0.04190966,
0.02975568]], dtype=float32),
     array([1., 1., 1.], dtype=float32)]
```

Revenons à XOR Problem

Les étapes à suivre pour créer le modèle



1. **Data**
2. **Model**
   - Create the model, compile the model: keras, tensorflow
     - Optimizer function: adam
     - Loss function: binary_crossentropy
   - Train the model
   - Parameters
3. **Prediction**
4. **Evaluation**



1. Dataset

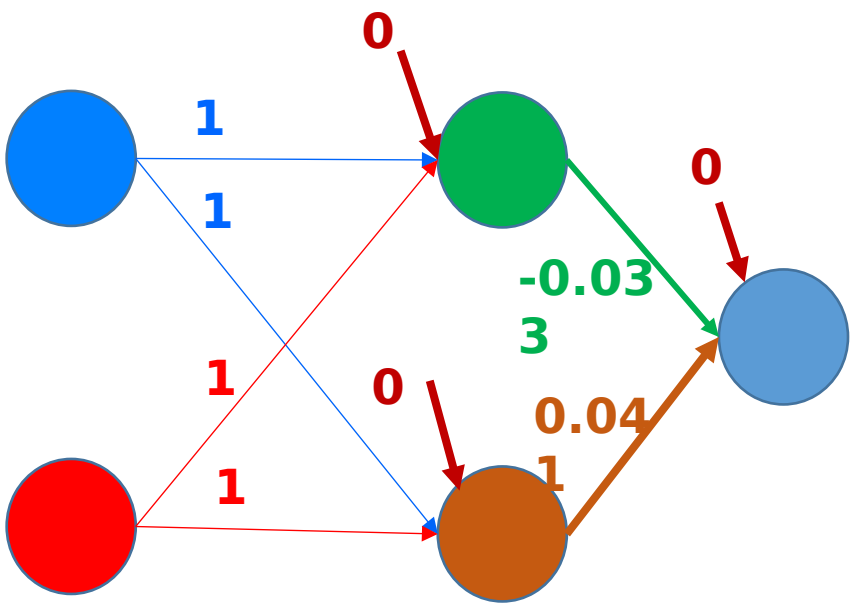
```
X=np.array([[0,0],[0,1],[1,0],[1,1]])
y = np.array(np.logical_xor(X[:, 0] > 0, X[:, 1] >
0), dtype=int)
```

2. Model

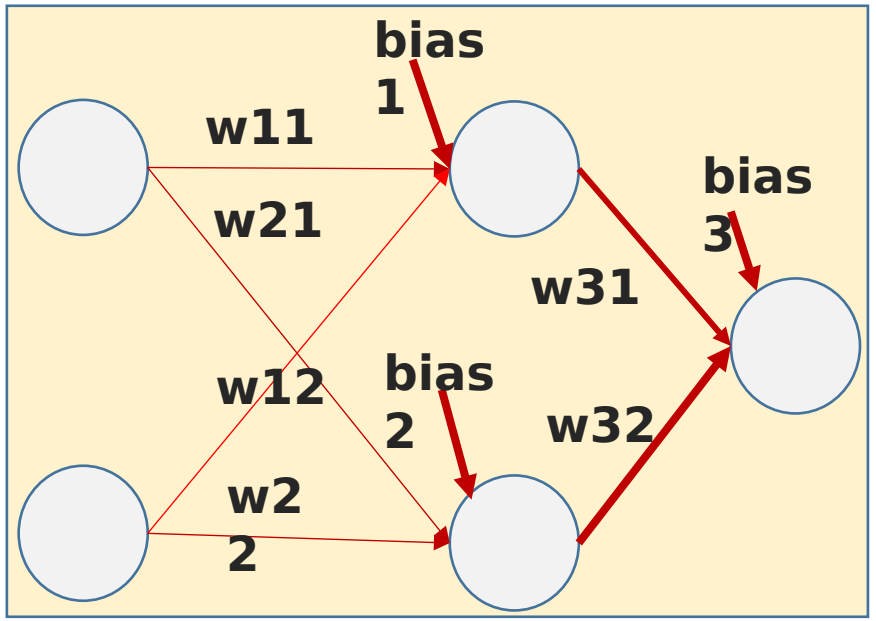
```
from keras.models import Sequential

from keras.layers import Activation, Dense,Flatten

model = Sequential()

#input_layer = Flatten(input_shape=[2,1])

hidden_layer = Dense(2,input_dim=2,
kernel_initializer='ones',bias_initializer='zeros', activation='relu')

output_layer =
Dense(1,kernel_initializer='random_uniform',bias_initializer='zero
s', activation='sigmoid')

#model.add(input_layer)

model.add(hidden_layer)

model.add(output_layer)

coef=model.get_weights()

print(coef)
```

```
model.compile(loss='binary_crossentropy',optimizer='adam',metri
cs=['accuracy'])
```

```
model.fit(X,y,epochs=1000)

print(model.get_weights())

print(model.predict(X)
```



[array([[1., 1.], [1., 1.]], dtype=float32), array([0., 0.], dtype=float32), array([[-0.03330588], [0.04190588]], dtype=float32), array([0.], dtype=float32)]


bias
1
w11
w21
bias
3
w31
w12
bias
2
w32
w2
2

```
model.fit(X,y,epochs=1000)
print(model.get_weights())
print(model.predict(X)
```

2. Model (avec une autre fonction d'activation)

```
from keras.layers import LeakyReLU
act = LeakyReLU(alpha = 0.3)
```

```
model = Sequential()
hidden_layer = Dense(2,input_dim=2, kernel_initializer='ones',bias_initializer='zeros',activation=act)
model.add(hidden_layer)
output_layer = Dense(1,kernel_initializer='random_uniform',bias_initializer='zeros',activation='sigmoid')
model.add(output_layer)
model.add(hidden_layer)
model.add(output_layer)
coef=model.get_weights()
print(coef)
```

```
model.compile(loss='binary_crossentropy',optimizer='adam',metri
cs=['accuracy'])
```

```
model.fit(X,y,epochs=1000,batch_size
=4)

print(model.get_weights())

print(model.predict(X))
```